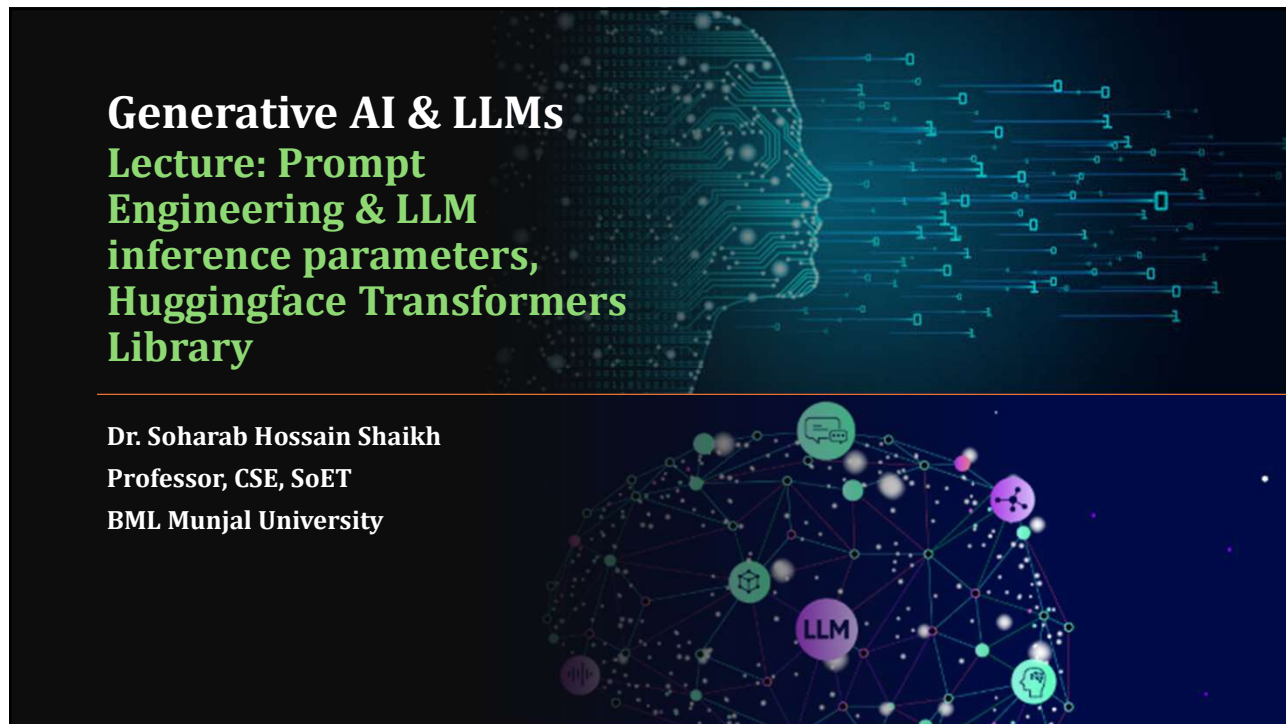




1

Agenda

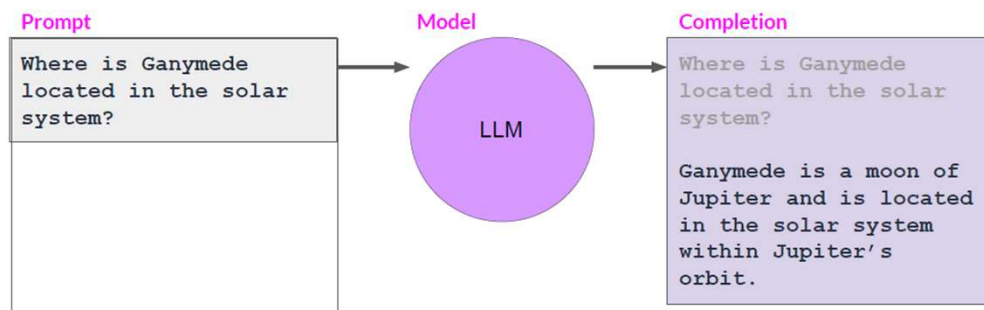
- Prompt & Completion
- In-context Learning (ICL) – zero-shot, one-shot- few-shot
- LLM Parameters – temperature, top_k, top_p
- Huggingface Transformers Library

2

Prompting & Prompt Engineering

3

Prompting and prompt engineering

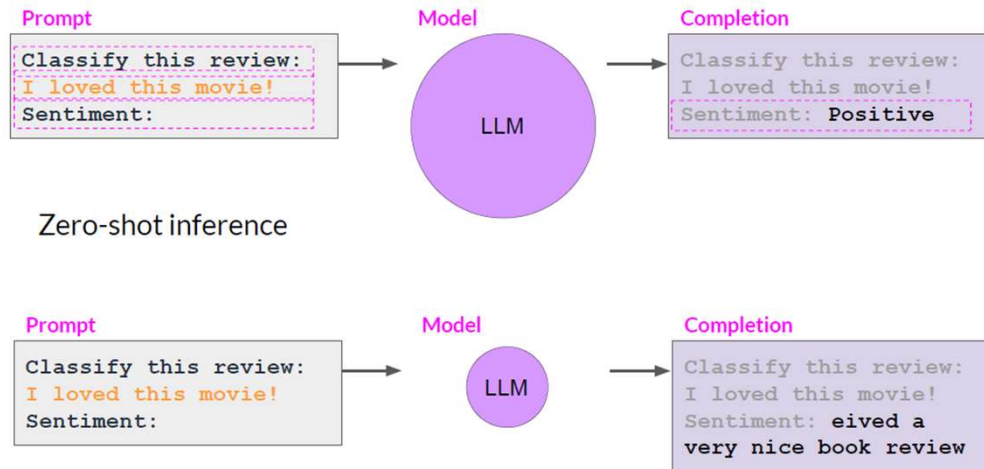


Context window: typically a few thousand words

Slide Credits: DeepLearning.ai course: Generative AI and LLMs with AWS
<https://www.deeplearning.ai/courses/generative-ai-with-llms/>

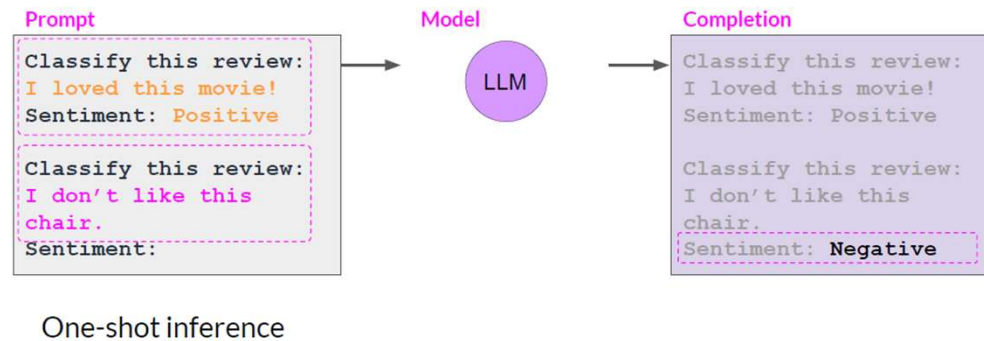
4

In-context learning (ICL) - zero shot inference



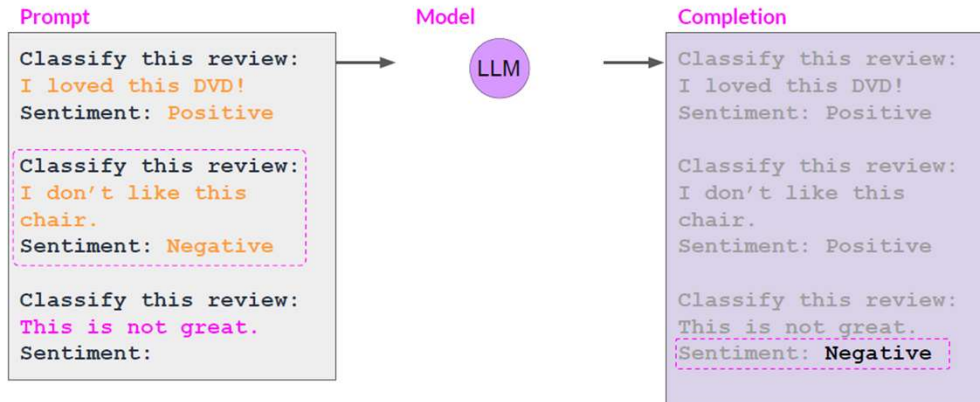
5

In-context learning (ICL) - one shot inference



6

In-context learning (ICL) - few shot inference



7

Summary of in-context learning (ICL)



8

Generative Configurations Parameters for Inference

9

Generative configuration - inference parameters

Enter your prompt here...

Max new tokens 200

Sample top K 25

Sample top P 1

Temperature 0.8

Submit

Inference configuration parameters

10

Generative configuration - max new tokens

Enter your prompt here...

Max new tokens 200

Sample top K 25

Sample top P 1

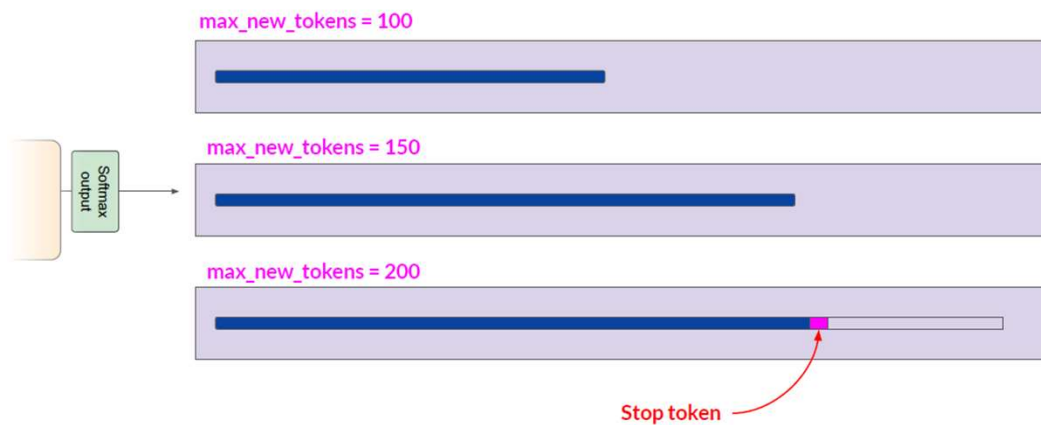
Temperature 0.8

Submit

Max new tokens

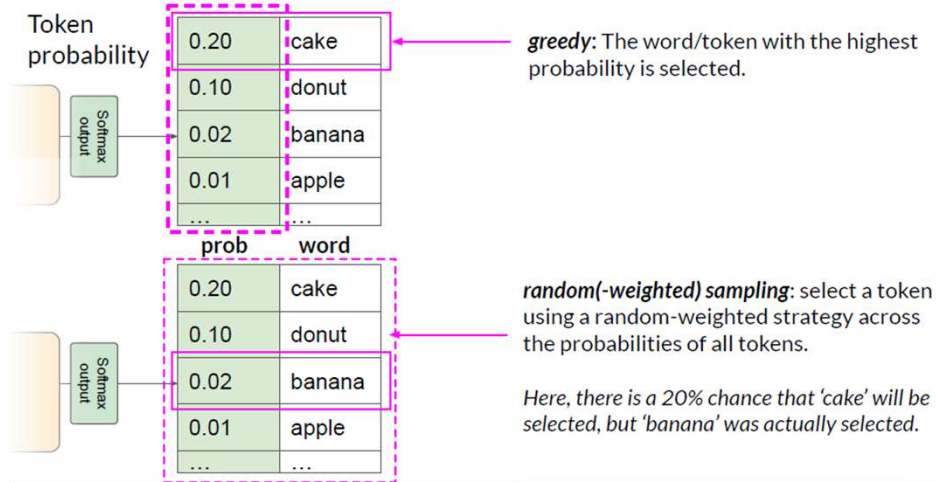
11

Generative config - max new tokens



12

Generative config - greedy vs. random sampling



13

Generative configuration - top-k and top-p

The interface shows a text input field on the left labeled 'Enter your prompt here...'. To the right are several configuration sliders and buttons:

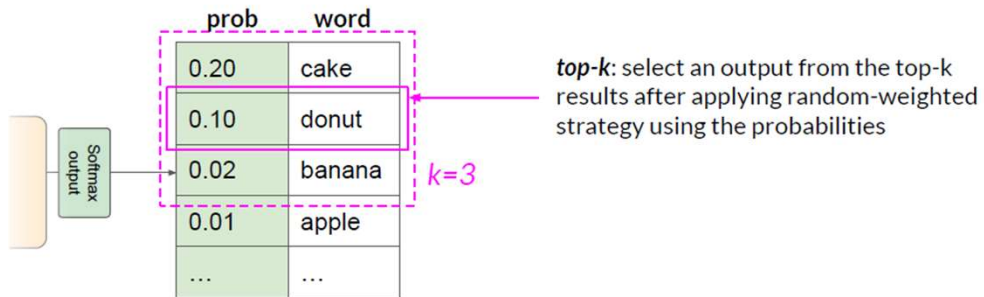
- Max new tokens:** A slider set to 200.
- Sample top K:** A slider set to 25.
- Sample top P:** A slider set to 1.
- Temperature:** A slider set to 0.8.
- Submit:** A button at the bottom right.

A pink dashed box highlights the 'Sample top K' and 'Sample top P' sliders, indicating the focus of the configuration.

Top-k and top-p sampling

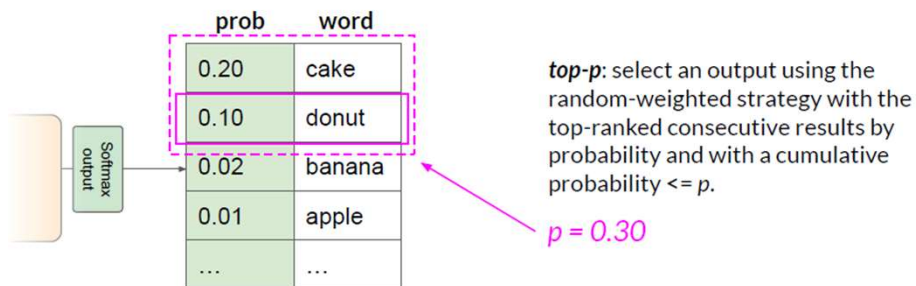
14

Generative config - top-k sampling



15

Generative config - top-p sampling



16

Generative configuration - temperature

Enter your prompt here...

Max new tokens 200

Sample top K 25

Sample top P 1

Temperature 0.8

Submit

Temperature

17

Generative config - temperature

To implement temperature scaling, we modify the standard Softmax function by dividing the raw scores (logits) by a temperature parameter T .

The probability P_i for a specific token i is calculated as:

$$P_i = \frac{\exp(z_i/T)}{\sum_{j=1}^K \exp(z_j/T)}$$

Variable Definitions

- z_i : The **logit** (raw score) for the i -th token in the vocabulary.
- T : The **Temperature** parameter.
- K : The total number of tokens in the model's vocabulary.
- $\exp$: The exponential function (e^x).

18

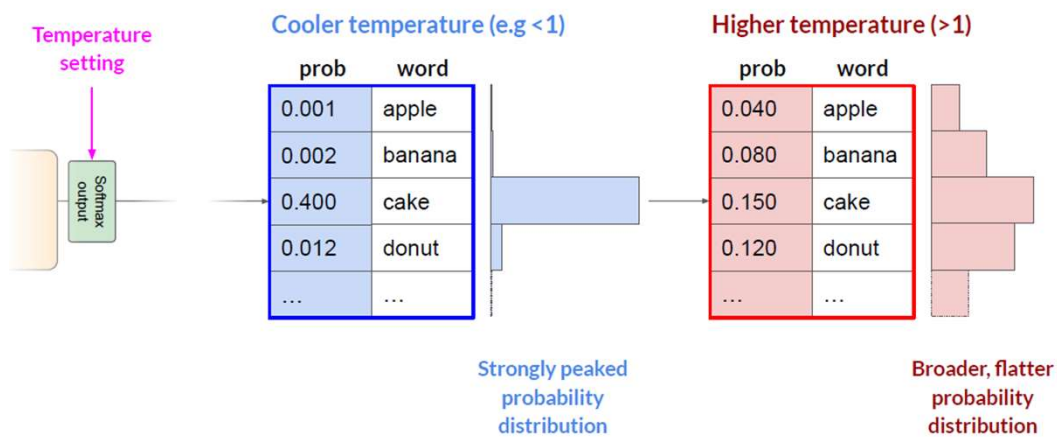
Generative config - temperature

How the Formula Affects Output

1. **When $T = 1$:**
The formula is identical to the standard Softmax. The model's original confidence levels are preserved.
2. **When $T \rightarrow 0$ (Low Temperature):**
The differences between logits are amplified. The highest logit z_{max} dominates the fraction, making P_{max} approach **1.0** and all other probabilities approach **0**. This results in "Greedy" behavior.
3. **When $T > 1$ (High Temperature):**
The logits are pushed closer together. The "gap" between the most likely and least likely word shrinks, creating a **flatter probability distribution**. This makes the model more likely to pick "surprising" words, which we perceive as creativity.

19

Generative config - temperature



20

Generative config - temperature

Temperature	Label	Effect on Model	Vibe
T<1.0	Cool	Increases the gap. The top choice becomes much more likely than the others.	Conservative: Focused, factual, but can be repetitive.
T=1.0	Neutral	The original distribution is preserved.	Balanced: The "intended" behavior of the model.
T>1.0	Hot	Decreases the gap. Low-probability words get a higher chance of being picked.	Creative: Diverse, unexpected, but prone to "hallucinations" or gibberish.

21



Huggingface Transformers

State-of-the-art pretrained models for inference and training

22

Huggingface Transformers

- Hugging Face Transformers is an open-source Python library that provides access to thousands of pre-trained Transformers models for natural language processing (NLP), computer vision, audio tasks, and more.
- It simplifies the process of implementing Transformer models by abstracting away the complexity of training or deploying models in lower level ML frameworks like PyTorch, TensorFlow and JAX.
- Transformers acts as the model-definition framework for state-of-the-art machine learning with text, computer vision, audio, video, and multimodal models, for both inference and training.
- Transformers works with **Python 3.10+**, and **PyTorch 2.4+**.

<https://github.com/huggingface/transformers>

23

Huggingface Transformers

Models Pretrained weights + code (Llama 3, DBRX, BERT, GPT-2, etc.)	Supported Frameworks ✓ PyTorch ✓ TensorFlow ✓ Flax / JAX
Tokenizers Model-specific text preprocessing (Python & Rust)	
Pipelines One-line inference for common NLP tasks	
Datasets Standard dataset loading & preprocessing	
Trainer Training loop abstraction with logging & checkpointing	Project Types this helps with: 1. Applying pretrained models to new tasks 3. Analyzing model behavior & representations

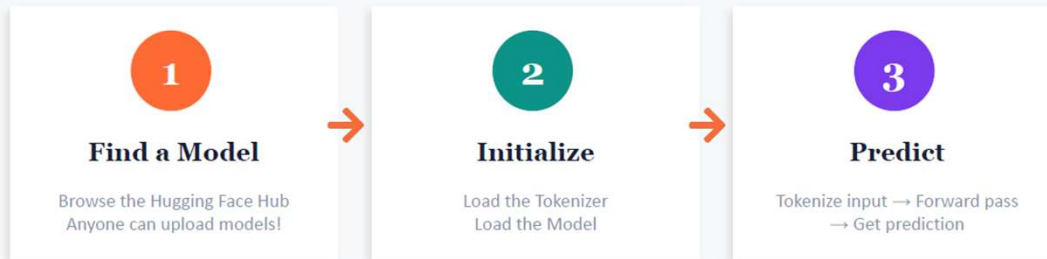
24

How to use?



`pip install transformers`

HF workflow follows this 3-step pattern:



```

tokenizer = AutoTokenizer.from_pretrained("siebert/sentiment-roberta-large-english")
model = AutoModelForSequenceClassification.from_pretrained(...)
outputs = model(**tokenizer(text, return_tensors="pt"))
  
```

25

Tokenizers: From Text to Numbers



Three Ways to Load a Tokenizer

DistilBertTokenizer	Model-specific, Python
DistilBertTokenizerFast	Model-specific, Rust (faster)
AutoTokenizer	Auto-detects model type ◆ Recommended

26

Tokenizer: Step by Step

```
input_tokens = tokenizer.tokenize(input_str)
# → ["Hu", "##gging", "Face", "Transformers", "is", "great", "!"]

input_ids = tokenizer.convert_tokens_to_ids(input_tokens)
# → [20164, 10932, 10289, 25267, 1110, 1632, 106]

input_ids_special = cls + input_ids + sep
# → [101, 20164, ..., 106, 102] ([CLS]...[SEP])

decoded = tokenizer.decode(input_ids_special)
# → "[CLS] Hugging Face Transformers is great! [SEP]"
```

- ✓ **Special Tokens** [CLS] at start, [SEP] at end — model-dependent
- ✓ **Subword Tokenization** "Hugging" → "Hu" + "##gging" — handles unknown words
- ✓ **Attention Mask** 1 for real tokens, 0 for padding — tells model what to attend to

27

Padding, Truncation & Batching

```
model_inputs = tokenizer(
    ["Short text", "A much longer text..."],
    return_tensors="pt", padding=True, truncation=True
)
```

What happens:

Sentence 1:	[CLS]	Short	text	[SEP]	[PAD]	[PAD]	[PAD]
Sentence 2:	[CLS]	A	much	longer	text	...	[SEP]
Attn Mask:	1	1	1	1	0	0	0

← Sentence 1's attention mask (padding = 0)

28

Models: Architecture & Heads

Three Types of Transformer Models

Encoders

BERT, DistilBERT

Classification, NER, analysis

Decoders

GPT-2, LLaMA

Text generation

Enc-Dec

BART, T5

Summarization, translation

Task-Specific Model Heads

***Model**

Base representations

***ForMaskedLM**

Fill in the blank

***ForSequenceClassification**

Sentiment, topic classification

***ForTokenClassification** NER, POS tagging

***ForQuestionAnswering** Extractive QA

***ForCausalLM**

Text generation

29

Model Inference: It's Just PyTorch!

```
# Pass inputs via keyword args or dict unpacking
model_inputs = tokenizer(text, return_tensors="pt")
model_outputs = model(**model_inputs)
```

```
# Calculate loss normally
loss = F.cross_entropy(outputs.logits, labels)
loss.backward() # Standard PyTorch!
```

Standard nn.Module

Use any PyTorch optimizer, scheduler, or loss function

Built-in Loss

Pass 'labels' kwarg and the model computes loss automatically

Attention & Hidden States

Set output_attentions=True and output_hidden_states=True

30

Fine-tuning: Loading Datasets

IMDB Dataset

25,000 train / 25,000 test reviews
Binary sentiment: POSITIVE / NEGATIVE
We use 128 train + 32 val (for speed)
Truncated to 50 tokens each

```
from datasets import load_dataset

ds = load_dataset("stanfordnlp/imdb")

# Tokenize the dataset
tokenized = ds.map(
    lambda ex: tokenizer(
        ex['text'], padding=True,
        truncation=True),
    batched=True)
```

Dataset Preprocessing Pipeline

1. Tokenize

Apply tokenizer to text with padding & truncation

2. Remove text

remove_columns(["text"]) — model only needs IDs

3. Rename label

rename_column("label", "labels") — HF convention

4. Set format

set_format("torch") — convert to PyTorch tensors

31

Fine-tuning: The Training Loop

Option A: PyTorch Training Loop

```
optimizer = AdamW(model.parameters(),
                  lr=5e-5)

for epoch in range(num_epochs):
    model.train()
    for batch in train_dataloader:
        output = model(**batch)
        output.loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    model.eval() # validation
    for batch in eval_dataloader:
        with torch.no_grad():
            output = model(**batch)
```

Option B: HF Trainer (recommended)

```
args = TrainingArguments(
    output_dir="checkpoints",
    per_device_train_batch_size=16,
    num_train_epochs=2,
    eval_strategy="epoch",
    learning_rate=2e-5)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=fn)

trainer.train()
```

32

HF Trainer: Callbacks & Evaluation

Callbacks

EarlyStoppingCallback

Stop training when validation loss plateaus

LoggingCallback (custom)

Log metrics to JSONL file on each logging step

Custom callbacks can hook into:

on_log, on_epoch_end, on_evaluate, on_save,
...

Evaluation

trainer.evaluate()

Returns evaluation metrics

trainer.predict(test_ds)

Returns predictions + metrics

compute_metrics function:

Custom metric calculation at eval time
(accuracy, F1, etc.)

Recommended Starting Hyperparameters

Epochs

2-4

Learning Rate

2e-5 or 5e-5

Batch Size

**As large as fits
GPU**

Weight Decay

0, 0.01, or 0.1

Warmup Steps

0, 100, or 500

33

Text Generation with GPT-2

```
from transformers import AutoModelForCausalLM, AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('gpt2')
model = AutoModelForCausalLM.from_pretrained('distilgpt2')

prompt = "Once upon a time"
tokenized = tokenizer(prompt, return_tensors="pt")

output = model.generate(**tokenized,
                        max_length=50, do_sample=True, top_p=0.9)
```

Key Generation Parameters

max_length	Maximum number of tokens to generate
do_sample	Enable sampling (vs greedy decoding)
top_p	Nucleus sampling — only consider top-p probability mass
temperature	Controls randomness: lower = more deterministic
top_k	Only sample from top-k most likely tokens

34

Pipelines & Masked Language Modeling

Pipelines: One-Line Inference

```
sa = pipeline("sentiment-analysis",
              model="siebert/...")
sa("This movie is great!")
```

Available pipelines: sentiment-analysis, fill-mask, text-generation, question-answering, summarization, translation, NER, ...

Masked Language Modeling

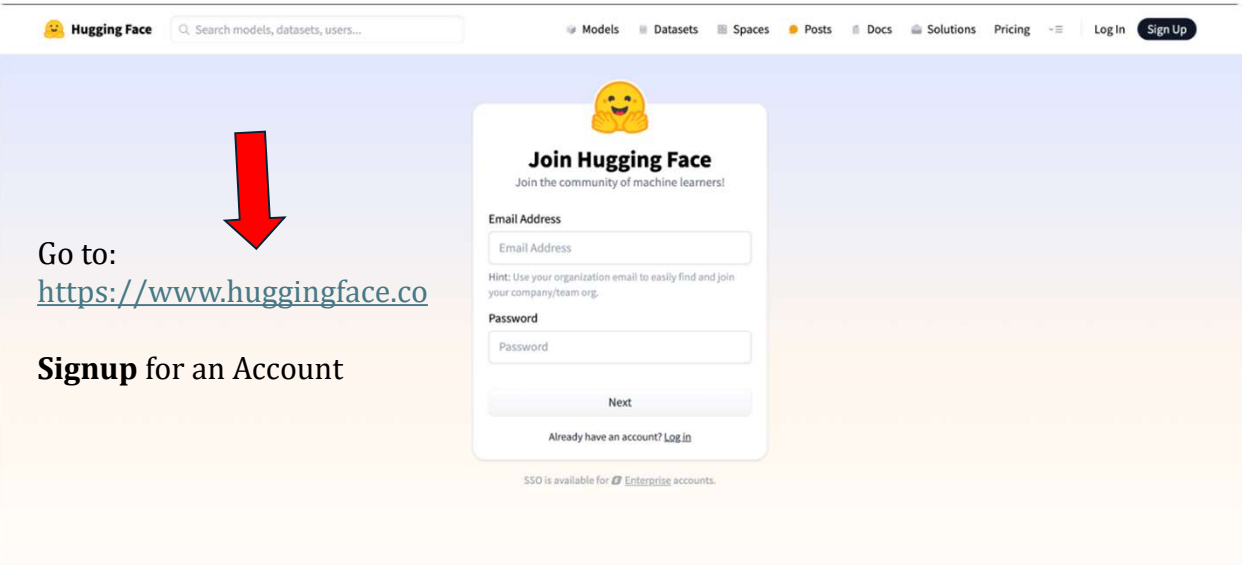
```
m1m = pipeline("fill-mask", "bert-base-cased")
m1m("I am [MASK] to learn!")
```

Top predictions:
 → excited (35.5%)
 → going (15.6%)
 → eager (7.9%)
 → here (3.6%)

Custom Datasets (Appendix 1 in notebook)

You can create custom PyTorch Datasets for HF models — just return tokenized dicts from `__getitem__`. See the E2E Dataset example in the notebook for encoder-decoder tasks.

35



Go to:
<https://www.huggingface.co>

Signup for an Account

Join Hugging Face
 Join the community of machine learners!

Email Address

Hint: Use your organization email to easily find and join your company/team org.

Password

Already have an account? [Log in](#)

SSO is available for [Enterprise](#) accounts.

36

Access Tokens

User Access Tokens

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions.

Do not share your Access Tokens with anyone; we regularly check for leaked Access Tokens and remove them immediately.

Name	Value	Last Refreshed Date	Last Used Date	Permissions
medQA	hf_...mIRF	Jul 28, 2025	Jul 28, 2025	FINEGRAINED
colab	hf_...fggK	Jan 15, 2025	Jan 15, 2025	FINEGRAINED
rp	hf_...Mzbr	Jan 14, 2025	Jan 14, 2025	FINEGRAINED
agents	hf_...VzUC	Jul 26, 2024	21 days ago	READ

- Go to Profile >> Settings
- Select Access Tokens tab
- Create New Token (HF_TOKEN)
- Save the token for later use

37

Why should you use Transformers?

1. Easy-to-use state-of-the-art models:

- High performance on natural language understanding & generation, computer vision, audio, video, and multimodal tasks.
- Low barrier to entry for researchers, engineers, and developers.
- Few user-facing abstractions with just three classes to learn.
- A unified API for using all provided pretrained models.

2. Lower compute costs, smaller carbon footprint:

- Share trained models instead of training from scratch.
- Reduce compute time and production costs.
- Dozens of model architectures with 1M+ pretrained checkpoints across all modalities.

3. Choose the right framework for every part of a model's lifetime:

- Train state-of-the-art models in 3 lines of code.
- Move a single model between PyTorch/JAX/TF2.0 frameworks at will.
- Pick the right framework for training, evaluation, and production.

4. Easily customize a model or an example to your needs:

- We provide examples for each architecture to reproduce the results published by its original authors.
- Model internals are exposed as consistently as possible.
- Model files can be used independently of the library for quick experiments.

38

References

1. DeepLearning.ai course: Generative AI and LLMs with AWS

<https://www.deeplearning.ai/courses/generative-ai-with-llms/>

2. Huggingface Course on Youtube (Playlist)

https://www.youtube.com/watch?v=00GKzGyWFEs&list=PLo2EIpl_JMQvWfQndUesu0nPBAZ9gP1o

3. Total noob's intro to Hugging Face Transformers

https://huggingface.co/blog/noob_intro_transformers

4. Transformers Notebooks

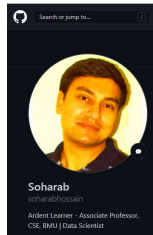
<https://huggingface.co/docs/transformers/en/notebooks>

<https://github.com/huggingface/notebooks>

39



40



Thank You

Class material will be uploaded here:
https://github.com/soharabhossain/GenAI_n_LLMs